EMERGEX

Campus-Linked Student Accommodation Search and Viewing-Request System

Phase 4 — System Design

IT Project 700 · Semester 2, 2026
Richfield Graduate Institute of Technology · Faculty of Information Technology
Group 55 · Supervisor: Thabiso (ThabisoA@richfield.ac.za)
Due 30 October 2026

Group members

Student no.	Name	Surname	Email	Contact no.
402308936	Tristan	Moses	402308936@my.richfield.ac.za	082 801 5082
402411668	Reenesh	Ramawthar	402411668@my.richfield.ac.za	084 604 1521
402411227	Joyce	Kamalu	402411227@my.richfield.ac.za	067 107 5365
402412304	Novithan	Naidoo	402412304@my.richfield.ac.za	062 082 9186
402413409	Thingo	Shezi	402413409@my.richfield.ac.za	071 429 9251
402111846	Lungelo	Ngwane	402111846@my.richfield.ac.za	064 253 4563

Contents

Contents

EMERGEX 1
Contents 2
4.1 Introduction 3
4.1.1 Purpose of the design phase 3
4.1.2 Relationship to Phase 3 3
4.1.3 Design corrections carried forward 4
4.2 System Design — Description of the Proposed System 6
4.2.1 Overview 6
4.2.2 Design objectives 7
4.2.3 Design principles 8
4.2.4 System boundary and actors 9
4.2.5 Design constraints 9
4.2.6 Key design decisions 11
4.3 Architectural Design 12
4.3.1 Software architectural design 12
4.3.2 Module structure and dependencies 13
4.3.3 Request lifecycle 14
4.3.4 Hardware architectural design 14
4.3.5 Network architectural design 15
4.3.6 Class diagram 16
4.4 Physical Design 18
4.4.1 Directory structure 18
4.4.2 File inventory 19
4.4.3 Configuration and environment 20
4.4.4 Storage estimates 21
4.5 Database Design 22
4.5.1 Entity-relationship diagram 22
4.5.2 Normalisation 22
4.5.3 Table specifications 23

4.5.4 Referential integrity rules 27
4.5.5 Indexing strategy 28
4.5.6 Portability — MySQL and SQLite 29
4.6 Program Design 30
4.6.1 Viewing-request validation and capture 30
4.6.2 Filtered search 34
4.6.3 Move-in cost calculation 36
4.6.4 Distance and travel time 38
4.6.5 The data-source seam 39
4.7 Interface Design 40
4.7.1 Menu and navigation design 40
4.7.2 Input design 41
4.7.3 Output design 43
4.7.4 Accessibility and responsive design 52
4.8 Security and Backup Design 52
4.8.1 Threat model 52
4.8.2 Security controls 54
4.8.3 Data protection and POPIA 55
4.8.4 Backup and recovery design 56
4.8.5 Outstanding security work 57
Conclusion 58
References 58

4.1 Introduction

4.1.1 Purpose of the design phase

Phase three created an understanding of what EMERGEX must do. This phase establishes how the high does it. The set of buildable specifications produced is an architecture, physical layout, database design, program logic, interface design, and security and backup design. This set would be a conversion of the functional requirements FR1-FR10, the non-functional requirements NFR1-NFR8 and the business rules BR-01 to BR-14 into a buildable specification.

The rationale behind design decisions is captured in documentation. If a design document only records the conclusion with no other rationale, it isn't very useful later on since the

reader doesn't know which parts are load-bearing and which were arbitrary. The statement mentions about a rejected consideration made in the past.

4.1.2 Relationship to Phase 3

During Phase 3, we produced the refined entity-relationship model, data dictionary for captured data, context and Level 1 data-flow diagrams, and business-rule catalogue. These types of inputs are provided at this stage and not things which were to re-derive and therefore referenced. The Phase 3 data model is extended in Section 4.5 to a complete physical database design with data types, constraints and indexes, and deletion behaviour.

Table 4.1.1 — Phase 3 outputs used in this phase

Phase 3 output	Where it is developed in Phase 4
FR1–FR10 functional requirements	4.2.1 overview; 4.6 program design; 4.7 interface design
NFR1–NFR8 quality requirements	4.3 architecture; 4.7.4 accessibility; 4.8 security
BR-01 to BR-14 business rules	4.5.4 integrity rules; 4.6 program design
Refined ERD (nine tables)	4.5.1 and 4.5.3, extended to a physical design
Data dictionary for `viewing_requests`	4.5.3, with types, nullability and defaults
§3.4.5 findings (six integrity gaps)	4.1.3 below, and resolved in 4.5.4 and 4.6.1

4.1.3 Design corrections carried forward

Phase 3 §3.4.5 identified six rules the prototype does not fully enforce, and recorded that they are carried into Phase 4 as design corrections. Each is addressed in this document, and the design section that resolves it is named here so the thread from analysis to design is explicit.

Table 4.1.2 — Resolution of the Phase 3 findings

No.	Finding from Phase 3	Design response	Section
1	A viewing request can be saved with no rooms selected (BR-06 unenforced)	The validation sequence in 4.6.1 adds a check on the submitted room list before the transaction is opened	4.6.1
2	Selected rooms are not checked against the selected campus (BR-07 partial)	Each submitted slug is resolved and its `campus_id` compared with the submitted campus; a mismatch is a validation error	4.6.1
3	`move_in_month` and campus accepted without checking against allowed options	Both are validated against generated allow-lists — the twelve-month window and the campus table	4.6.1
4	`status` is free text with no database restriction	The column becomes `ENUM('new','contacted','viewed','closed')` in the physical design	4.5.3
5	Deleting a listing removes it from past viewing requests (`ON DELETE CASCADE`)	The constraint on `viewing_request_rooms.listing_slug` changes to `ON DELETE RESTRICT`; listings are removed by soft delete only	4.5.4

6	Full-name length is not checked against the 160-character column	A server-side maximum-length check is added to the validation sequence	4.6.1

A seventh item, not listed as an integrity finding but raised in Phase 3 §3.6.1, is also resolved here: the form currently tells the student that a parent or guardian "receives the same confirmation", while no email is sent. Under NFR5 (Honesty) the design in 4.7.2 specifies the corrected wording, and 4.8.5 records email delivery as outstanding work rather than an implemented feature.

4.2 System Design — Description of the Proposed System

4.2.1 Overview

EMERGEX is a server-rendered web application that replaces steps 2 to 5 of the current informal accommodation search (Phase 3 §3.3.2) with a single site. A student selects a Richfield campus, compares structured listings with walking and driving times and an inspection status, estimates the full move-in cost, and submits one tracked viewing request that may name a parent or guardian.

The system has no user accounts. This is a deliberate scope decision carried from Phase 2: the viewing request is a single anonymous transaction, and requiring registration before a student can ask to see a room would add a barrier to the one action the system exists to encourage. The shortlist, which would ordinarily need an account to persist, is kept in the browser instead.

Table 4.2.1 — Functional requirements and the components that satisfy them

Req.	Requirement	Realised by
FR1	Display every campus with city, province, confirmed status and bed count	`index.php` → `all_campuses()` → `repo_campuses()`
FR2	Search listings by campus, maximum rent, room type and amenities	`search.php` → `search_listings()` → `repo_search()`

FR3	Show rent, room type, inspected status, amenities and travel times	`listing.php`, `listing-card.php`, `listing_walk()`, `listing_drive()`
FR4	Plot listings and their campus on an interactive map	`search.php` → Leaflet in `app.js`
FR5	Shortlist up to four listings, kept across reloads	`app.js` with browser `localStorage`; `footer.php` drawer
FR6	Accept a viewing request and persist it	`apply.php` → `repo_save_viewing_request()`
FR7	Validate all input server-side	The validation sequence in `apply.php` (4.6.1)
FR8	Estimate total move-in cost	`calculate_payment()` in PHP; mirrored in `app.js` for live updates
FR9	Protect the form against cross-site request forgery	Session token compared with `hash_equals()`
FR10	Keep working with no database connected	The seam in `functions.php` (4.6.5)

4.2.2 Design objectives

1. **Deployability without ceremony.** The system must run on ordinary shared hosting with PHP and nothing else — no build step, no package manager, no compiled assets (NFR1).
2. **Correct data or no data.** A request is stored complete with the rooms it names, or not at all. Partial records are worse than none, because staff would act on them.

3. **Truthful presentation.** Nothing displayed may overstate what the system knows — unconfirmed campus locations, uninspected listings and sample data are all labelled as such (NFR5).
4. **Minimal personal data.** Only what a viewing requires is collected, and nothing that would make the database attractive to steal (NFR7, BR-12).
5. **Survivable degradation.** A missing database, a missing internet connection or a missing optional PHP extension must each reduce function rather than break the site.

4.2.3 Design principles

Table 4.2.2 — Principles applied throughout the design

Principle	How it is applied
Single point of truth	One file creates database connections (`db.php`); one file holds every SQL statement (`repository.php`); one CSS block holds every colour. Changing any of these means editing one place.
Separation of concerns	Pages render. The repository queries. Helpers calculate. No page contains SQL and no query contains presentation.
Fail safe, not open	Where a control cannot be evaluated the system refuses rather than permits: an unparseable CSRF token rejects the submission; an unresolvable listing slug is discarded rather than stored.
Defence in depth	Input is validated in the browser for usability and again on the server for correctness. The server check is the control; the browser check is a convenience.
Derive, do not hardcode	Displayed statistics — total beds, inspected count, median walking time — are computed from the data on each request. A hardcoded figure drifts from reality and destroys the reader's trust in everything beside it.

Design for the absent dependency	The database, the internet and the mbstring extension are each optional. Each has a defined fallback rather than an error.

4.2.4 System boundary and actors

Table 4.2.3 — Actors

Actor	Type	Interaction with the system
Student	Primary	Browses campuses, searches and filters listings, views detail, shortlists, estimates cost, submits a viewing request
Parent or guardian	Secondary	Named on the request by email address only. Does not interact with the system directly; the system holds the address so staff can include them when following up
Staff / administrator	Supporting	Maintains campuses, listings, amenities and FAQs, and follows up requests. No administrative interface exists in this release; data is maintained through the seed script and the database
Accommodation provider	External	Outside the system boundary. Contacted by staff outside the application; receives no automated message

Two things sit deliberately *outside* the boundary. The system sends no email, so a parent or guardian learns of a request only when staff contact them. And the system takes no payment and holds no funds, so the cost calculator is an estimate presented to the user, never a transaction.

4.2.5 Design constraints

Table 4.2.4 — Constraints the design must respect

Constraint	Origin	Design consequence

PHP 8.0 or newer, no framework	NFR1, module requirements	Page-per-URL structure; `match` expression used in the calculator sets the minimum version
No build step or package manager	NFR1	Hand-written CSS and JavaScript; external libraries loaded from a CDN, not bundled
Must run on MySQL or SQLite	NFR3	PDO throughout; no driver-specific SQL; `INSERT IGNORE` avoided in favour of an existence check
mbstring may be absent	Portability finding	A `str_len()` wrapper falls back to `strlen()`
Richfield branding used without approval	Project boundary	Every page carries `noindex` and a not-affiliated footer notice; the site is not published publicly
Richfield students are not NSFAS-eligible	Phase 3 §3.1	No NSFAS badge, filter or claim anywhere; a single FAQ answer explains the ineligibility
Campus coordinates are approximations	Open client item	A `confirmed` flag drives a "Location to confirm" badge; distances are labelled estimates

4.2.6 Key design decisions

Table 4.2.5 — Decisions, alternatives and rationale

Decision	Alternative considered	Rationale
Server-rendered pages, one PHP file per URL	A single-page application with a JSON API	An SPA needs a build step and a package manager, which NFR1 excludes, and would place three layers of indirection between a request and the code that answers it. The content is documents, not an application state machine.
Procedural PHP with a repository module	Object-oriented layer with entity classes and a data-mapper	At nine tables and four pages the ceremony would exceed the benefit. The discipline an OO design would enforce — one place for queries, one place for connections — is achieved here by module boundaries and is checked by convention markers (`[DB-*]`) that can be found with a single search.
Shortlist in `localStorage`	Server-side shortlist in the `shortlists` table	With no accounts there is no stable identity to key a server-side shortlist to, and issuing a tracking cookie to do it would collect personal data for a convenience feature. The table exists in the schema for when logins are added.
Graceful fallback to sample data	Fail loudly when the database is unavailable	The repository is shared between six students; a teammate without MySQL running must still be able to open the site. A `strict` configuration flag exists to invert this once the database is the source of truth.
Straight-line (haversine) distance	Street routing through a maps API	A routing API means an API key, a per-request cost, a network dependency and a rate limit. The interface labels the figures as

		estimates, which is honest and sufficient for comparing listings against one another.
Rent stored as whole rand in an integer column	`DECIMAL(10,2)`	The calculator rounds to whole rand at every step; storing cents would introduce drift between the stored figure and the displayed one without adding precision anyone uses.

4.3 Architectural Design

4.3.1 Software architectural design

The system uses a **four-layer architecture**. Each layer may call the layer below it and never the layer above. The strict rule that makes this real rather than aspirational is that *no page contains SQL* and *no file except the connector creates a database connection*.

1 · Presentation layer Server-rendered pages — one PHP file per URL. Renders HTML, holds no SQL. index.php search.php listing.php apply.php partials 2 · Domain / service layer Business logic and the data-source seam. Decides database or sample data. functions.php — lookups calculate_payment() haversine_km() e() escaping 3 · Data-access layer Every SQL statement in the project. Prepared statements only. repository.php [DB-QUERIES] db.php [DB-CONNECTOR] — the only new PDO 4 · Storage layer MySQL / MariaDB (9 tables) SQLite (single file) data.php — sample arrays (fallback) Figure 4.1 — Software architecture. The dashed path is the fallback taken when no database is configured or reachable.

Table 4.3.1 — Layer responsibilities

Layer	Responsibility	Must not
Presentation	Receive the request, call the domain layer, render HTML, escape every dynamic value	Contain SQL, open a connection, or perform business calculations
Domain / service	Business rules, calculations, and the decision of where data comes from	Produce HTML or know which database driver is in use

Data access	Every SQL statement; connection lifecycle; transactions	Contain presentation logic or business rules
Storage	Persist data and enforce integrity constraints	—

Why the seam matters. The domain layer exposes five lookup functions — `all_campuses()`, `all_listings()`, `all_amenities()`, `all_room_types()` and `all_faqs()` — and every page reads data through those and nothing else. Each one decides internally whether to call the repository or return the sample array. Because both paths return an identically shaped array, the entire presentation layer is unaware of which source is live. That single design choice is what makes FR10 (working with no database) a two-line branch rather than a parallel codebase.

4.3.2 Module structure and dependencies

Dependencies run in one direction only. There is no cycle anywhere in the graph, which is what allows any single module to be understood without reading the others.

index.php search.php listing.php apply.php functions.php header.php · footer.php listing-card.php repository.php data.php db.php config.php (git-ignored) PDO → MySQL / SQLite fallback Figure 4.2 — Module dependency graph. Solid arrows are required dependencies; dashed arrows are conditional. No cycles exist.
The convention markers embedded in the source make each junction findable. A single search for `[DB-` lists every point at which the application touches data, which is the review path a maintainer or an assessor would follow.

Table 4.3.2 — Convention markers in the source

Marker	File	Marks
`[DB-CONNECTOR]`	`db.php`	The only place `new PDO` appears
`[DB-QUERIES]`	`repository.php`	Every SQL statement

`[DB-WRITE]`	`apply.php`, `repository.php`	The single data-capture point
`[DB-3]`	`functions.php`	The read seam — database or sample, one branch
`[DB-4]`	`functions.php`	Aggregates behind the home-page statistics

4.3.3 Request lifecycle

The viewing-request submission is the only write path and exercises every layer. Figure 4.3 shows the sequence, including the two points at which the request can be refused.

Student apply.php functions.php repository.php Database POST form data + CSRF token verify token reject — "Your session expired" validate fields redisplay with field errors, entries kept resolve campus + slugs generate ref RF-YYMMDD-XXXX save_viewing_request() BEGIN INSERT viewing_requests INSERT viewing_request_rooms × n COMMIT new request id confirmation page showing the reference Figure 4.3 — Sequence diagram: submitting a viewing request. Both inserts occur inside one transaction, so a request can never be stored without the rooms it names (BR-11).

4.3.4 Hardware architectural design

The system is a conventional two-tier web application and makes no unusual hardware demands. Two deployment configurations are designed for: the development and assessment environment, and a notional production deployment.

(a) Development and assessment — single machine Developer workstation · Windows 11 · 8 GB RAM Apache (XAMPP) PHP 8.x module MySQL / MariaDB port 3306 Browser — Chromium / Firefox (b) Production — shared hosting Shared LAMP host Apache + PHP 8 TLS terminated here MySQL instance least-privilege user Nightly mysqldump → off-host storage Client devices Mobile phone360–414 px Tablet768–1024 px Desktop1280–1920 px Minimum server specification: 1 vCPU · 1 GB RAM · 500 MB disk. The application is stateless between requests apart from the PHP session, so capacity scales by adding web hosts behind a load balancer without redesign. Figure 4.4 — Hardware deployment, development and production.

Table 4.3.3 — Hardware specification

Component	Development / assessment	Production (recommended)

Web server	Apache 2.4 via XAMPP on the developer's machine	Apache 2.4 or nginx, 1 vCPU, 1 GB RAM
Runtime	PHP 8.0+ as an Apache module	PHP 8.2+ with OPcache enabled
Database	MariaDB via XAMPP, port 3306 (3307 if the port is taken)	MySQL 8 on the same host or a managed instance
Storage	≈ 50 MB	500 MB, allowing for growth and backups
Client	Any current browser; tested on Chromium	Same; no plug-ins or minimum specification

4.3.5 Network architectural design

During development, application is accessible using HTTP but in production, it has to be served using HTTPS since the viewing-request form contains personal details. The browser contacts both external services never the server and both have a defined degraded mode.

Student's browser localStorage: shortlist session cookie Web server Apache + PHP 8 port 80 / 443 only /public content served Database server MySQL · TCP 3306 not exposed publicly HTTPS HTML PDO OpenStreetMap tiles via Leaflet CDN Google Fonts web font files browser-side only Degraded mode: with no internet the page still renders — system fonts replace the web fonts and the map area stays empty. No personal data is sent to either third party. Network topology and external dependencies Solid arrows: requests. Dashed: responses and optional dependencies. Figure 4.5 — Network architecture. The server makes no outbound connections; the only external calls are made by the browser.

Table 4.3.4 — Network specification

Path	Protocol / port	Carries	If unavailable

Browser → web server	HTTPS / 443 (HTTP 80 in development)	Page requests; the viewing-request POST, which carries personal data	No service — this is the critical path
Web server → database	TCP / 3306, loopback or private network	Prepared statements and result sets	Site falls back to sample data; form validates but does not store
Browser → OpenStreetMap	HTTPS / 443	Map tiles only; no user data	Map area stays empty; the keyboard-navigable listing list remains
Browser → Google Fonts	HTTPS / 443	Font files only	System font stack is used; layout is unaffected

As a result, we have two design points at the network level. In the first place, the server establishes no outbound connections. Thus, the host requires no egress rules and there is not any third-party service whose outage can bring the app down. In addition, we only allow the browser to make external calls which don't contain user data. Therefore, we don't transfer any personal information over a boundary the project doesn't control – which is the position from where POPIA compliance will be easiest to defend.

4.3.6 Class diagram

> **A note on the class model.** The implementation is procedural PHP: `functions.php`, `repository.php` and `db.php` are modules of functions, not classes. The diagram below is therefore a *conceptual class model* — it shows the entities the procedural code manipulates, their attributes and their relationships, and the operations currently implemented as functions are listed against the entity they act on.
>
> This is stated plainly rather than presented as a diagram of classes that exist in the source, because the two are not the same thing. Figure 4.2 is the diagram of what the code is actually structured as. If the system were migrated to an object-oriented design, Figure 4.6 is the model it would adopt.

Campus - id : String «PK» - name : String - city : String - province : String - lat, lng : Decimal - confirmed : Boolean - beds : Integer + totalBeds() Listing - slug : String «PK» - name : String - area : String - lat, lng : Decimal - rent : Integer (ZAR) - beds : Integer - verified : Boolean - isActive : Boolean + walkMinutes() + driveMinutes() RoomType - id : String «PK» - label : String Amenity - id : String «PK» - short : String - label : String

ViewingRequest - id : Integer «PK» - reference : String «UQ» - fullName : String - email : String - phone : String - guardianEmail : String [0..1] - preferredDate : Date - moveInMonth : String - notes : String [0..1] - status : Status - createdAt : Timestamp + generateReference() + validate() «enumeration» Status new contacted viewed closed Shortlist - ownerKey : String - listingSlug : String - createdAt : Timestamp browser-held until logins Faq - id : Integer «PK» - question : String - answer : Text 10..* serves 0..*1 0..*0..* has 10..* receives 11..* names Figure 4.6 — Conceptual class model. Dashed entities are designed but not active in this release: `Shortlist` is held in the browser until accounts exist.

Table 4.3.5 — Relationships and their multiplicities

Relationship	Multiplicity	Design note
Campus serves Listing	1 : 0..*	Every listing belongs to exactly one campus (BR-04). Enforced by a NOT NULL foreign key.
Listing has RoomType	0..* : 1	One room type per listing, drawn from a controlled vocabulary.
Listing has Amenity	0..* : 0..*	Many-to-many, resolved by the `listing_amenities` link table with a composite primary key that makes duplicates impossible.
Campus receives ViewingRequest	1 : 0..*	The request records the campus the student attends, which is how staff route it.
ViewingRequest names Listing	1 : 1..*	Resolved by `viewing_request_rooms`. The 1..* minimum is the design correction for Phase 3 finding 1 — the current build permits zero.
Shortlist references Listing	0..* : 1	Designed, not active. Held in `localStorage` until accounts exist.

4.4 Physical Design

Physical design specifies how the logical design is laid out as files and directories on a server, how configuration is supplied, and what storage the system consumes.

4.4.1 Directory structure

```
Richfield accommodation interface/
├── index.php                  Landing page — all ten sections
├── search.php                 Search, filters and map
├── listing.php                Room detail (?slug=)
├── apply.php                  Viewing-request form, validation and
save
├── README.md
├── .gitignore                 Blocks credentials and database dumps
│
├── includes/
│   ├── db.php                 [DB-CONNECTOR] the only `new PDO`
│   ├── repository.php         [DB-QUERIES] every SQL statement
│   ├── config.sample.php      Credentials template — committed
│   ├── config.php             Real credentials — NEVER committed
│   ├── data.php               Sample data: fallback source and seed
source
│   ├── functions.php          Distance, money, formatting, the read
seam
│   ├── header.php             Shared page shell and navigation
│   ├── footer.php             Shared footer and shortlist drawer
│   └── listing-card.php       Reusable card partial
│
├── database/
│   ├── schema.mysql.sql       Tables for MySQL / MariaDB
```

```
│   ├── schema.sqlite.sql     Tables for SQLite
│   ├── seed.php              Creates tables and loads reference
data
│   └── richfield.sqlite      Created at runtime — NEVER committed
│
└── assets/
    ├── css/style.css         The entire design system
    └── js/app.js             Shortlist, calculator, map, reveals
```

A deployment weakness to record. In this layout the application files sit in the web root, so `includes/` and `database/` are inside the directory the server publishes. They are protected only by the fact that a direct request to a PHP file there produces no output, and by `.gitignore` keeping credentials out of the repository.

The correct production layout moves everything except the four page files and `assets/` *above* the document root, so the server has no path to them at all. This is recorded as outstanding work in 4.8.5 rather than presented as implemented.

4.4.2 File inventory

Table 4.4.1 — Deployed components

Component	Files	Responsibility	Changes when
Page scripts	4	Receive requests and render HTML	A page is added or its layout changes
Shared partials	3	Page shell, navigation, footer, listing card	Site-wide chrome changes
Domain and data modules	4	Logic, queries, connection, sample data	A rule, query or entity changes

Database scripts	3	Schema for two drivers, plus the seeder	The data model changes
Static assets	2	Design system and client behaviour	Visual or interaction changes
Configuration	1 + 1	Committed template; local real credentials	Deployment environment changes

4.4.3 Configuration and environment

Configuration is supplied by a single file, `includes/config.php`, which returns an associative array. It is deliberately absent from version control; `config.sample.php` is committed as the template and is copied on each deployment.

Table 4.4.2 — Configuration keys

Key	Values	Effect
`driver`	`mysql` · `sqlite`	Selects the storage engine. SQLite needs no server and keeps the whole database in one file.
`host`, `port`	Host, 3306 / 3307	MySQL only. Port 3307 is used where another MySQL already holds 3306.
`database`	`richfield_accommodation`	Schema name, created by the schema script itself.
`username`, `password`	Credentials	Should be a least-privilege account in production — SELECT, INSERT and UPDATE only, never DROP.

`sqlite_path`	Filesystem path	SQLite only. The directory is created if absent.
`strict`	`true` · `false`	When true a failed connection throws instead of falling back to sample data. Intended to be true once the database is the source of truth.

The absence of a configuration file is itself a valid state and selects the sample-data path. This is what allows a new team member to clone the repository and open the site without any setup at all.

4.4.4 Storage estimates

Table 4.4.3 — Storage projection

Item	Unit size	At launch	At 5 000 requests
Application source	—	≈ 400 KB	≈ 400 KB
Reference data (9 tables)	—	< 100 KB	< 500 KB
`viewing_requests`	≈ 400 bytes/row	0	≈ 2 MB
`viewing_request_rooms`	≈ 140 bytes/row	0	≈ 1 MB (avg 1.5 rooms)
Listing photography (when supplied)	≈ 200 KB each	0	≈ 4 MB for 18 listings
Total		**< 1 MB**	**< 10 MB**

Storage is not a constraint at any realistic volume. The operative limit is the POPIA retention period agreed for captured personal data (4.8.3), which caps growth of the two captured tables regardless of demand.

4.5 Database Design

4.5.1 Entity-relationship diagram

Nine tables, in two groups: six holding reference data maintained by staff, and three holding data captured from students.

REFERENCE DATA — maintained by staff campuses PK id name, city, province lat, lng confirmed beds, sort_order room_types PK id label, sort_order amenities PK id short, label sort_order listings PK slug FK campus_id FK room_type name, area, blurb lat, lng rent, beds verified, is_active seed, image_url, timestamps listing_amenities PK listing_slug PK amenity_id faqs PK id question, answer, is_active CAPTURED DATA — written by students · personal information viewing_requests PK id UQ reference FK campus_id full_name, email, phone guardian_email preferred_date move_in_month notes status «enum» created_at viewing_request_rooms PK request_id PK listing_slug shortlists PK id UQ owner_key + slug FK listing_slug Figure 4.7 — Entity-relationship diagram, crow's foot notation. `shortlists` is dashed: designed and present in the schema, but unused until student accounts are added.

4.5.2 Normalisation

The design is in **Third Normal Form**. The progression is shown below because the two link tables are the product of it rather than an arbitrary choice.

Table 4.5.1 — Normalisation

Form	Requirement	How the design satisfies it
1NF	Atomic values; no repeating groups	A listing's amenities would naturally be a list. Storing them as a comma-separated column would break 1NF and make "find rooms with backup power" a `LIKE` scan. They are a separate table instead.
2NF	1NF, and no partial dependency on part of a composite key	The two composite-key tables (`listing_amenities`, `viewing_request_rooms`) hold *only*

		their key columns, so no non-key attribute can depend on part of the key.
3NF	2NF, and no transitive dependency between non-key attributes	A listing stores `campus_id`, not the campus name, city or province — those depend on the campus, not on the listing. Likewise `room_type` rather than the room-type label.

One deliberate denormalisation is absent. While every listing card shows walking and driving times, they are not stored anywhere. The listing and campus coordinate pair is computed on every request to the server. When you store them, it would denormalise what is going to be a derived value and when you correct the coordinate, it will be wrong in two places. The cost is eighteen haversine calculations per page, which is negligible; if the catalogue grew to thousands the design would cache the values not store them as columns.

4.5.3 Table specifications

Table 4.5.2 — campuses

Column	Type	Null	Notes
`id`	VARCHAR(40) PK	No	Slug, e.g. `bryanston`
`name`, `city`, `province`	VARCHAR(120)	No	
`lat`, `lng`	DECIMAL(10,7)	No	Suburb-level approximation, not a surveyed position
`confirmed`	TINYINT(1) default 0	No	0 = location came from a directory. Drives the "Location to confirm" badge (BR-03)

`beds`	INT UNSIGNED default 0	No	Feeds the computed total on the home page
`sort_order`	INT default 0	No	Fixed display order independent of name

Table 4.5.3 — listings

Column	Type	Null	Notes
`slug`	VARCHAR(120) PK	No	URL key; a natural key, so the address is readable and stable
`name`	VARCHAR(160)	No	Invented building names only — never a real business
`area`	VARCHAR(160)	No	Suburb level only, never a street address (BR-14, POPIA)
`campus_id`	VARCHAR(40) FK	No	→ `campuses.id`, ON UPDATE CASCADE
`room_type`	VARCHAR(40) FK	No	→ `room_types.id`, ON UPDATE CASCADE
`lat`, `lng`	DECIMAL(10,7)	No	Required by FR3 and FR4
`rent`	INT UNSIGNED	No	**Whole rand, never cents.** The calculator rounds to whole rand throughout; decimals introduce drift

`verified`	TINYINT(1) default 0	No	1 = physically inspected. Drives the "Inspected" badge (BR-02)
`is_active`	TINYINT(1) default 1	No	Soft delete — hide a listing without losing request history (BR-01)
`image_url`	VARCHAR(500)	Yes	NULL selects the generated placeholder panel
`created_at`, `updated_at`	TIMESTAMP	No	Server-set; `updated_at` auto-updates

Table 4.5.4 — viewing_requests (captured personal data)

Column	Type	Null	Notes and rule
`id`	INT UNSIGNED AI PK	No	Internal identifier, never shown
`reference`	VARCHAR(20) UNIQUE	No	`RF-YYMMDD-XXXX`, shown to the student (BR-10)
`full_name`	VARCHAR(160)	No	Minimum 2 characters; **maximum 160, newly enforced** (finding 6)

`email`	VARCHAR(190)	No	Validated address
`phone`	VARCHAR(20)	No	South African mobile: 0 or +27, then 6/7/8, then 8 digits (BR-08)
`guardian_email`	VARCHAR(190)	Yes	Optional; validated when supplied; stored NULL when blank
`campus_id`	VARCHAR(40) FK	No	**Validated against the campus table, newly enforced** (finding 3)
`preferred_date`	DATE	No	Not in the past (BR-09)
`move_in_month`	CHAR(7)	No	`YYYY-MM`; **validated against the 12-month window** (finding 3)
`notes`	VARCHAR(500)	Yes	Maximum 500 characters; NULL when blank

`status`	**ENUM('new','contacted','viewed','closed')**	No	**Changed from VARCHAR(20)** — finding 4. Default `new`
`created_at`	TIMESTAMP	No	Server-set. Also the basis for the retention policy

The remaining five tables are specified in Phase 3 §3.4.3 and are unchanged except where Table 4.1.2 records a correction.

4.5.4 Referential integrity rules

Deletion behaviour is chosen per relationship rather than applied uniformly, because the consequences differ. The guiding question is: *if the parent row disappears, is the child row meaningless or is it evidence?*

Table 4.5.5 — Foreign keys and deletion behaviour

Constraint	On delete	Reasoning
`listings.campus_id` → campuses	RESTRICT (implicit)	A campus with listings cannot be removed. Campuses are stable reference data.
`listings.room_type` → room_types	RESTRICT (implicit)	As above; the vocabulary is controlled.
`listing_amenities.*`	CASCADE	The link row is meaningless without either parent. Correct to remove it.

`viewing_requests.campus_id` → campuses	RESTRICT (implicit)	A request must always retain the campus it was made for.
`viewing_request_rooms.request_id`	CASCADE	Deleting a request must take its room rows with it — this is also the POPIA deletion route.
`viewing_request_rooms.listing_slug`	**RESTRICT** (was CASCADE)	**Phase 3 finding 5.** Under CASCADE, removing a listing silently erased it from every historical request — so a student's record of what they asked to view would change retrospectively. Listings are now removed by soft delete (`is_active = 0`) only, and the database refuses a hard delete that would destroy history.
`shortlists.listing_slug`	CASCADE	A saved room that no longer exists is not evidence of anything.

4.5.5 Indexing strategy

Indexes are placed on the columns the application actually filters and sorts by, which is determined from the queries in `repository.php` rather than guessed.

Table 4.5.6 — Indexes

Index	Table	Query it serves

`idx_listings_campus`	listings	Every search — `WHERE campus_id = ?` is in all of them
`idx_listings_rent`	listings	The maximum-rent filter (FR2)
`idx_listings_active`	listings	`is_active = 1`, present in every listing read (BR-01)
`uq_reference`	viewing_requests	Enforces BR-10 uniqueness and serves the collision check
`idx_vr_campus`, `idx_vr_status`, `idx_vr_created`	viewing_requests	The administrative queue, once an admin view is built
`idx_la_amenity`	listing_amenities	The amenity sub-select in the search query
`uq_owner_listing`	shortlists	Prevents the same room being saved twice by one owner

4.5.6 Portability — MySQL and SQLite

NFR3 requires the same code to run on either engine. Three design decisions make that work, and each came from a real incompatibility.

Table 4.5.7 — Portability measures

Issue	Design response
Foreign keys off by default in SQLite	`PRAGMA foreign_keys = ON` is issued immediately after connecting, otherwise CASCADE rules are silently ignored and the two engines behave differently.

`INSERT IGNORE` / `INSERT OR IGNORE` differ	Neither is used. An existence check precedes the insert, which is slightly more work and identical on both engines.
Bound integers arrive as strings	With emulation disabled, a bound value reaches the engine as a string. SQLite compares `COUNT(...)` against `'2'` and never matches, while MySQL coerces and works — an engine-specific bug that only appears on one of them. The amenity `HAVING` count is therefore interpolated as a cast integer, which is safe because it is `count()` of a PHP array and can never contain user input. Every actual value remains bound.

4.6 Program Design

Pseudo code for the five algorithms that carry business meaning. Presentation and simple reads are omitted. Where a step implements a correction from Table 4.1.2 it is marked **[NEW]**.

4.6.1 Viewing-request validation and capture

The system's only write path, and the one place a student's data enters. The order matters: the token is checked before anything else, and the transaction is not opened until every rule has passed.

```
PROCEDURE HandleViewingRequest(postData, session)


  // ---- 1. Cross-site request forgery (FR9) ----------------------
---

  IF NOT ConstantTimeEquals(session.csrf, postData.csrf) THEN

      RETURN Error("Your session expired. Please try again.")

  END IF


  errors  ← empty map

  values  ← Trim(each field of postData)

  slugs   ← Intersect(postData.listings, KnownListingSlugs())
```

```
  // ---- 2. Field validation (FR7, BR-06 to BR-09) ----------------
---

  IF Length(values.fullName) < 2 THEN

      errors.fullName ← "Enter your full name"

  ELSE IF Length(values.fullName) > 160 THEN                    //
[NEW] finding 6

      errors.fullName ← "Name must be 160 characters or fewer"

  END IF



  IF NOT IsValidEmail(values.email) THEN

      errors.email ← "Enter a valid email address"

  END IF



  // BR-08 — strip whitespace first so "082 123 4567" is accepted

  IF NOT Matches(StripSpaces(values.phone), "^(\+27|0)[6-8][0-
9]{8}$") THEN

      errors.phone ← "Enter a South African mobile number"

  END IF



  IF values.guardianEmail ≠ "" AND NOT
IsValidEmail(values.guardianEmail) THEN

      errors.guardianEmail ← "Enter a valid email address"

  END IF



  IF values.date = "" THEN

      errors.date ← "Choose a preferred date"

  ELSE IF values.date < Today() THEN                            //
BR-09

      errors.date ← "Choose a date that is not in the past"
```

```
  END IF


  IF values.campus NOT IN KnownCampusIds() THEN                  //
[NEW] finding 3
      errors.campus ← "Choose a campus from the list"
  END IF


  IF values.moveIn NOT IN UpcomingMonths(12) THEN                //
[NEW] finding 3
      errors.moveIn ← "Choose a month from the list"
  END IF


  IF Length(values.notes) > 500 THEN
      errors.notes ← "Keep it under 500 characters"
  END IF


  IF Count(slugs) = 0 THEN                                       //
[NEW] BR-06, finding 1
      errors.listings ← "Select at least one room to view"
  END IF


  FOR EACH slug IN slugs DO                                      //
[NEW] BR-07, finding 2
      IF GetListing(slug).campus ≠ values.campus THEN
          errors.listings ← "All selected rooms must be at your
chosen campus"
          BREAK
      END IF
  END FOR
```

```
  IF errors is not empty THEN
      RETURN RedisplayForm(values, errors)   // entries preserved
  END IF

  // ---- 3. Reference generation (BR-10) ---------------------------
  attempts ← 0
  REPEAT
      reference ← "RF-" + FormatDate(Today(), "ymd") + "-" +
RandomHex(4)
      attempts  ← attempts + 1
  UNTIL NOT ReferenceExists(reference) OR attempts = 5

  // ---- 4. Persist atomically (BR-11) -----------------------------
  IF DatabaseAvailable() THEN
      TRY
          BEGIN TRANSACTION
              requestId ← INSERT INTO viewing_requests (…, status =
'new')
              FOR EACH slug IN slugs DO
                  INSERT INTO viewing_request_rooms (requestId,
slug)
              END FOR
          COMMIT
          stored ← TRUE
      CATCH error
          ROLLBACK
```

```
          LogError(error)                          // never shown to
the student

          RETURN RedisplayForm(values,

                   "We could not save your request just now.")

      END TRY

  ELSE

      stored ← FALSE                               // FR10 — validated,
not saved

  END IF



  RETURN Confirmation(reference, stored)

END PROCEDURE
```

Two design points worth drawing out.

Errors are collected, not thrown. Every field is checked even after one has failed, so the student sees every problem at once rather than fixing them one submission at a time. The submitted values are preserved on redisplay.

The confirmation tells the truth about what happened. The `stored` flag drives the wording: "saved and with the accommodation provider" when a row exists, "no database is connected, so this request was not saved" when it does not. Under NFR5 the system must never claim a request was stored when it was not.

4.6.2 Filtered search

Runs in SQL when a database is connected and in PHP over the sample arrays when not. Both paths return the same shape, so the page is unaware of which ran.

```
FUNCTION SearchListings(campusId, maxRent, roomTypes[],
wantedAmenities[])



  IF DatabaseReady() THEN

      sql    ← "SELECT … FROM listings

                  WHERE campus_id = ? AND is_active = 1 AND rent <=
?"

      params ← [campusId, maxRent]
```

```
      IF roomTypes is not empty THEN
          sql    ← sql + " AND room_type IN (" +
Placeholders(roomTypes) + ")"
          params ← params + roomTypes
      END IF

      IF wantedAmenities is not empty THEN
          // AND, not OR — the interface says "Must include", so a
listing
          // must have every amenity selected, not merely one of
them.
          sql ← sql + " AND slug IN (
                           SELECT listing_slug FROM listing_amenities
                            WHERE amenity_id IN (" +
Placeholders(wantedAmenities) + ")
                            GROUP BY listing_slug
                           HAVING COUNT(DISTINCT amenity_id) = " +
Count(wantedAmenities) + ")"
          params ← params + wantedAmenities
      END IF

      rows ← ExecutePrepared(sql, params)
  ELSE
      rows ← Filter(SampleListingsFor(campusId), listing ⇒
                  listing.rent ≤ maxRent
                  AND (roomTypes is empty OR listing.room IN
roomTypes)
                  AND EveryOneOf(wantedAmenities) IN
listing.amenities)
```

```
  END IF


  // Closest on foot first, in both paths
  SortBy(rows, listing ⇒ WalkMinutes(listing))
  RETURN rows keyed by slug
END FUNCTION
```

The placeholder lists are built from the *count* of selected values, never from the values themselves, so the statement shape varies with the filters while every value remains bound. This is the design that satisfies NFR2 while still allowing a variable number of filters.

4.6.3 Move-in cost calculation

Uses BR-13. The most meaningful rule states that a discount applies to the rent only and never to the deposit and the administration fee.

```
FUNCTION CalculatePayment(monthlyRent, leaseMonths, depositMonths,
plan)


  CONSTANT UPFRONT_DISCOUNT ← 0.05
  CONSTANT TERMLY_DISCOUNT  ← 0.02
  CONSTANT MONTHS_PER_TERM  ← 3
  CONSTANT ADMIN_FEE        ← 950        // rand, fixed


  rent   ← Max(0, monthlyRent)
  months ← Max(0, leaseMonths)
  gross  ← rent × months


  rate ← CASE plan OF
            "upfront" : UPFRONT_DISCOUNT
            "termly"  : TERMLY_DISCOUNT
            OTHERWISE : 0.0
```

```
        END CASE

  discount  ← Round(gross × rate)
  rentTotal ← Round(gross − discount)

  // The deposit is refundable. Discounting it would misrepresent
  // what is actually being held, so the discount never touches it.
  deposit ← Round(rent × depositMonths)
  total   ← rentTotal + deposit + ADMIN_FEE

  instalments ← CASE OF
                  months = 0        : 0
                  plan = "upfront"  : 1
                  plan = "termly"   : Ceiling(months ÷
MONTHS_PER_TERM)
                  OTHERWISE         : months
                END CASE

  instalment ← IF instalments = 0 THEN 0
                                  ELSE Round(rentTotal ÷
instalments)

  dueNow ← IF plan = "upfront" THEN total
                               ELSE deposit + ADMIN_FEE + instalment

  RETURN { rentTotal, discount, deposit, admin: ADMIN_FEE,
           total, dueNow, instalment, instalments }
```

```
END FUNCTION
```

Table 4.6.1 — Worked example: R4 500/month, 12 months, 1 month deposit

Plan	Discount	Rent total	Instalments	Due now	Total
Monthly	R 0	R 54 000	12 × R 4 500	R 9 950	R 59 450
Termly (2%)	R 1 080	R 52 920	4 × R 13 230	R 18 680	R 58 370
Upfront (5%)	R 2 700	R 51 300	1	R 56 750	R 56 750

The monthly figures match the values recorded from the running prototype (R 9 950 due now, R 4 500 instalment, R 59 450 total), which confirms the specification above describes the implemented behaviour rather than an intended one.

4.6.4 Distance and travel time

```
CONSTANT EARTH_RADIUS_KM ← 6371

CONSTANT WALK_KMH        ← 5

CONSTANT DRIVE_KMH       ← 30


FUNCTION HaversineKm(lat1, lng1, lat2, lng2)

  dLat ← Radians(lat2 – lat1)

  dLng ← Radians(lng2 – lng1)

  h    ← Sin(dLat ÷ 2)² + Sin(dLng ÷ 2)² × Cos(Radians(lat1)) ×
Cos(Radians(lat2))

  RETURN 2 × EARTH_RADIUS_KM × ASin(Sqrt(h))

END FUNCTION
```

```
FUNCTION PaceMinutes(km, kmh)

  IF km ≤ 0 THEN RETURN 0 END IF

  RETURN Max(1, Round((km ÷ kmh) × 60))    // never report "0
minutes away"

END FUNCTION
```

The distance was measured by straight line not by routing on the street. This is done consciously as noted in Table 4.2.5. The reason behind the Max(1, …) guard is because a listing a hundred metres from campus would otherwise read as zero minutes, which looks like missing data rather than “very close”.

4.6.5 The data-source seam

The single branch that satisfies FR10. Each of the five lookup functions follows this pattern, and nothing above them knows which source answered.

```
FUNCTION AllListings()

  STATIC cache ← null                    // once per request, not
once per call



  IF cache = null THEN

      IF DatabaseReady() THEN            // connected AND schema
imported

          cache ← RepositoryListings()

      ELSE

          cache ← SampleListings()

      END IF



      IF cache is empty THEN

          cache ← SampleListings()       // connected but table
empty

      END IF

  END IF
```

```
  RETURN cache

END FUNCTION
```

Take note of the final guard. A connected database with an empty listings table would render a site with no content whatsoever – technically correct but useless. In this case, falling back keeps the site live and the sample data banner tells the viewer what he is seeing.

4.7 Interface Design

4.7.1 Menu and navigation design

The website contains four pages. The navigation consists of a single persistent header with six destinations and a shortlist control; there are no nested menus, as a four-page site does not necessitate a hierarchy and a student arriving under pressure should see all options at once.

There are four pages on the website. The navigation consists of a single persistent header containing six destinations and a shortlist control; there are no nested menus, because a four page site does not need a hierarchy, and nor should a student arriving under pressure not see all the options.

Table 4.7.1 — Navigation design decisions

Decision	Reasoning
Flat menu, no dropdowns	Four pages and five anchors. A hierarchy would hide options behind a hover, which is hostile on touch devices.
"Book a viewing" styled as the primary action on every page	It is the one action the system exists to produce. It is reachable in a single click from anywhere.

"For parents" as a first-class destination	Phase 3 W5 — the guardian pays but sees the costs last. Giving them a named section addresses the weakness in the navigation itself.
Shortlist as an overlay drawer, not a page	Comparing rooms should not lose the student's place in the results.
Skip-to-content link, first in tab order	Keyboard and screen-reader users bypass the header on every page (NFR4).

4.7.2 Input design

The viewing-request form is the only place the system accepts typed data. Every field is specified below with its validation, which is the contract between Figure 4.3 and the student.

Table 4.7.2 — Input specification (apply.php)

Field	Control	Req.	Validation	Error message
Full name	text	Yes	2–160 characters	"Enter your full name"
Email	email	Yes	Valid address format	"Enter a valid email address"
Phone	tel	Yes	0 or +27, then 6/7/8, then 8 digits; spaces stripped first	"Enter a South African mobile number"
Guardian email	email	No	Valid format when supplied	"Enter a valid email address"

Campus	select	Yes	Must be a known campus id **[NEW]**	"Choose a campus from the list"
Preferred date	date	Yes	Present or future; `min` set to today	"Choose a date that is not in the past"
Moving in	select	Yes	One of the next twelve months **[NEW]**	"Choose a month from the list"
Rooms to view	checkbox list	Yes	At least one; all at the chosen campus **[NEW]**	"Select at least one room to view"
Notes	textarea	No	Maximum 500 characters	"Keep it under 500 characters"
CSRF token	hidden	Yes	Constant-time match against the session	"Your session expired. Please try again."

Input design principles applied

- **Every field has a visible `<label>` bound by `for`/`id`** — placeholder text alone disappears on focus and is not announced reliably.
- **Errors appear beside the field**, not in a summary at the top, and the field carries `aria-invalid="true"`.
- **Submitted values are preserved on redisplay.** Clearing a form because one field failed is the most common way to lose a submission.
- **The form is marked `novalidate`** so the server's rules are what the student experiences. Browser validation otherwise silently blocks submissions the server has never seen, which makes the real rules untestable.
- **Rooms are pre-selected** when the student arrives from the shortlist or a listing page, so the common path requires no re-selection.

> **Corrected wording — NFR5.** The guardian-email hint currently reads "They receive the same confirmation and lease summary you do." No email is sent by this release, so the statement is untrue. The designed wording is:
>
> > *"Optional. We record it so the accommodation provider can include them when they get in touch."*

> The original wording may be restored once email delivery is implemented (4.8.5), and the two must change together.

4.7.3 Output design

Table 4.7.3 — Output specification

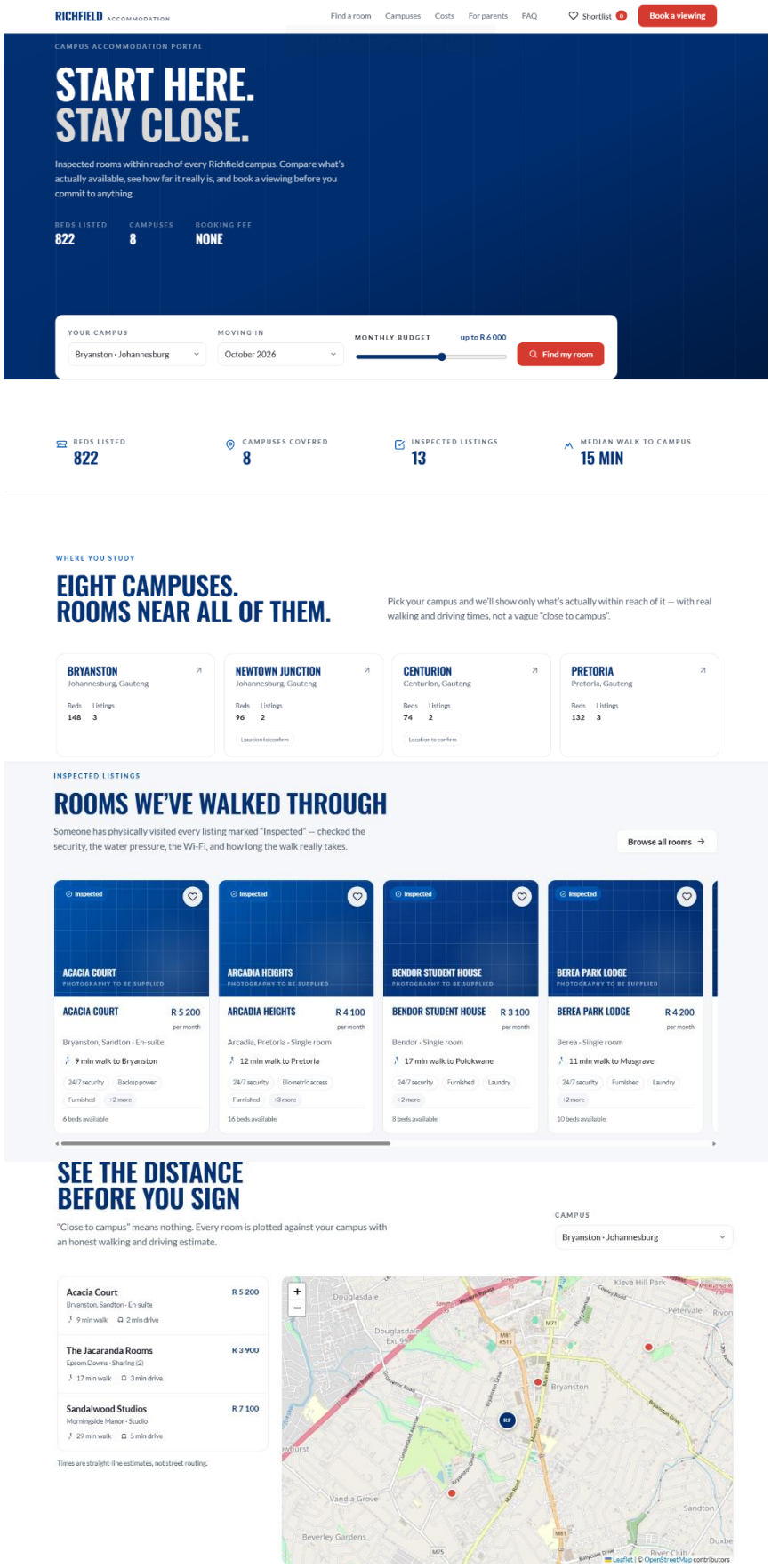
Output	Page	Content	Design rule
Campus cards	index.php	Name, city, province, bed count, confirmation badge	"Location to confirm" shown whenever `confirmed = 0` (BR-03)
Trust strip	index.php	Total beds, inspected count, median walking time	Computed per request, never hardcoded — a figure that contradicts the data below it destroys trust in the page
Search results	search.php	Listing cards sorted by walking distance ascending	Empty state with guidance copy when nothing matches, never a blank area
Map	search.php	Campus pin plus a pin per result	A keyboard-navigable list is the accessible equivalent (NFR4); OpenStreetMap attribution is a licence condition and must not be removed
Listing detail	listing.php	Rent, room type, amenities, inspected badge, walk and drive times, blurb	Unknown slug returns HTTP 404, not an empty page

Cost estimate	Costs section	Due now, instalment, instalments, total	All figures whole rand, formatted `R 4 500` with a space separator
Confirmation	apply.php	Reference, next steps, storage status	Wording follows what actually happened — never claims storage that did not occur
Sample-data banner	Every page	"Demonstration prototype — every listing shown is sample data"	Removed only by setting `SAMPLE_DATA` to false, once listings are real (NFR5)

Screen designs

The figures below are captures of the implemented interface, taken on 24 October 2026 from the application running under XAMPP with the MySQL schema imported.

Figure 4.9 — Landing page (index.php)



WHAT IT ACTUALLY COSTS

NO SURPRISES ON MOVE-IN DAY

Work out the full cost before you commit — deposit, admin fee and rent, in the payment rhythm that suits you.

Monthly | Per term | Upfront

Monthly rent R 3 800

Lease length: 12 months

Deposit: 1 month

YOUR ESTIMATE

Due before you move in

R 8 550

12 × monthly instalment	R 3 800
Deposit (refundable)	R 3 800
Once-off admin fee	R 950
Total over the lease	**R 50 350**

Estimate only. Final amounts are confirmed by the accommodation provider before you sign anything.

PAYING FOR IT

Richfield students are supported through flexible payment plans, internal bursary opportunities, and partnerships with private financial institutions for loan financing. Speak to your campus finance office about which of these applies to you.

THREE STEPS

FINDING A ROOM SHOULDN'T BE THE HARD PART OF YOUR DEGREE

01 SEARCH YOUR CAMPUS

Pick your campus and budget. You only ever see rooms that are genuinely within reach of where you study — with honest distances, not marketing language.

02 BOOK A VIEWING

Shortlist up to four and request viewings for all of them at once. It is free, it commits you to nothing, and the provider contacts you directly to confirm a time.

03 MOVE IN

Sign directly with the accommodation provider once you have seen the place. You know the full cost before you commit, because you worked it out here first.

FOR PARENTS AND GUARDIANS

YOU ARE PAYING FOR IT. YOU SHOULD SEE IT.

Most student accommodation is chosen in a hurry, from photos, by someone who has never signed a lease before. This is built so that does not have to happen.

Arrange a viewing together

Someone has been there

Inspected listings have been physically visited. We check secure entry, locks, hot water, electrics, backup power and measured internet speed — and we record what we found rather than summarising it into a score.

Plain-language leases

Every listing states the lease terms offered before you enquire: length, notice period, what happens over the December break, and whether the room can be sublet.

Deposits held by the provider

The deposit goes to the accommodation provider, not to Richfield and not to this platform. The listing states the amount and the refund conditions up front.

You are on the confirmation

Add your email to your student's viewing request and you receive the same confirmation and lease summary they do. No decisions happen without you seeing them.

QUESTIONS

STRAIGHT ANSWERS

Including the ones other platforms avoid.

Can I use NSFAS funding for this accommodation? ×

No. NSFAS funding is designated for public universities and TVET colleges, and Richfield is a private higher education institution, so Richfield students are not NSFAS-eligible. Richfield supports students through flexible payment plans, internal bursary opportunities, and partnerships with private financial institutions for loan financing. Your campus finance office can tell you which of these you qualify for.

Does Richfield run its own residences? +

What does an inspection actually check? +

How much deposit will I need? +

How long are the leases? +

What happens after I book a viewing? +

Is there a fee to use this service? +

How safe are these areas? +

What if something is wrong with a listing? +

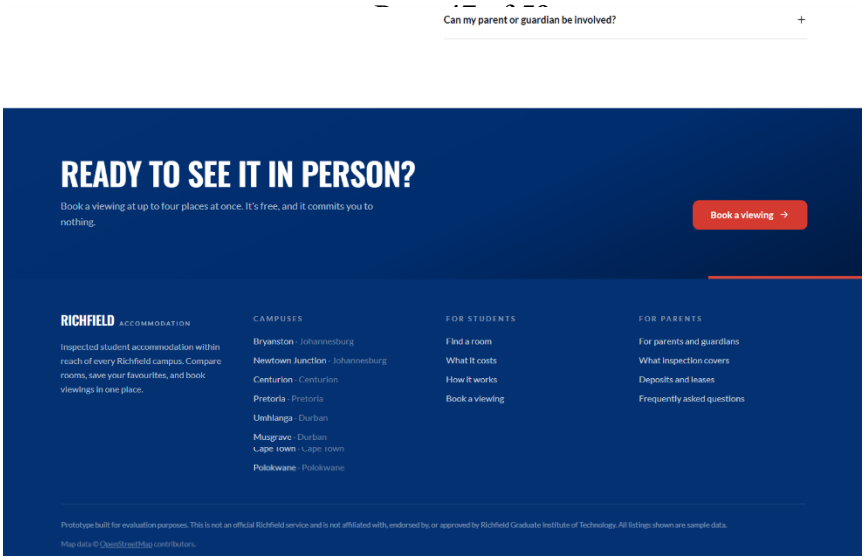


Figure 4.10 — Search page (search.php)

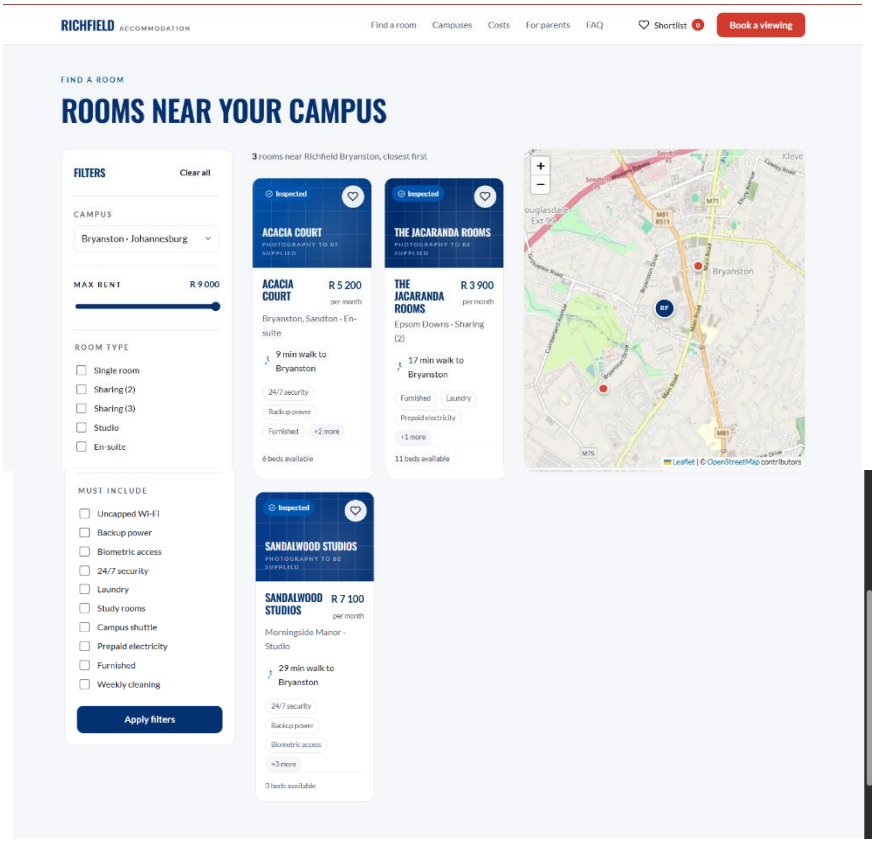


Figure 4.11 — Empty state

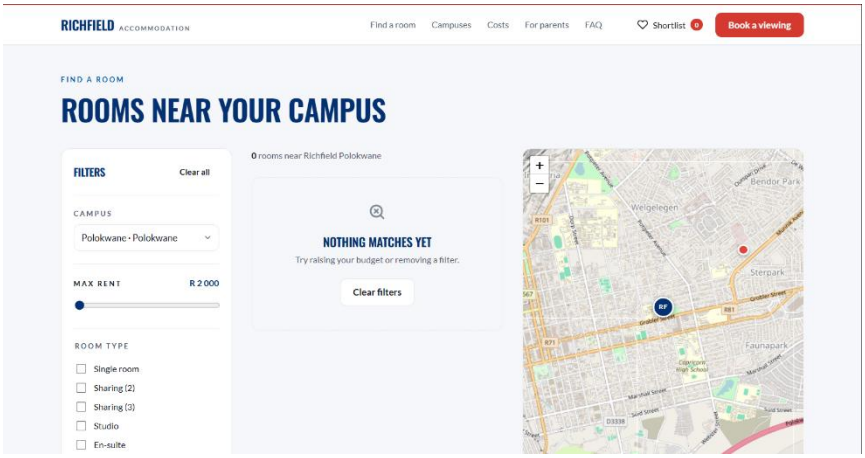


Figure 4.12 — Listing detail (listing.php)

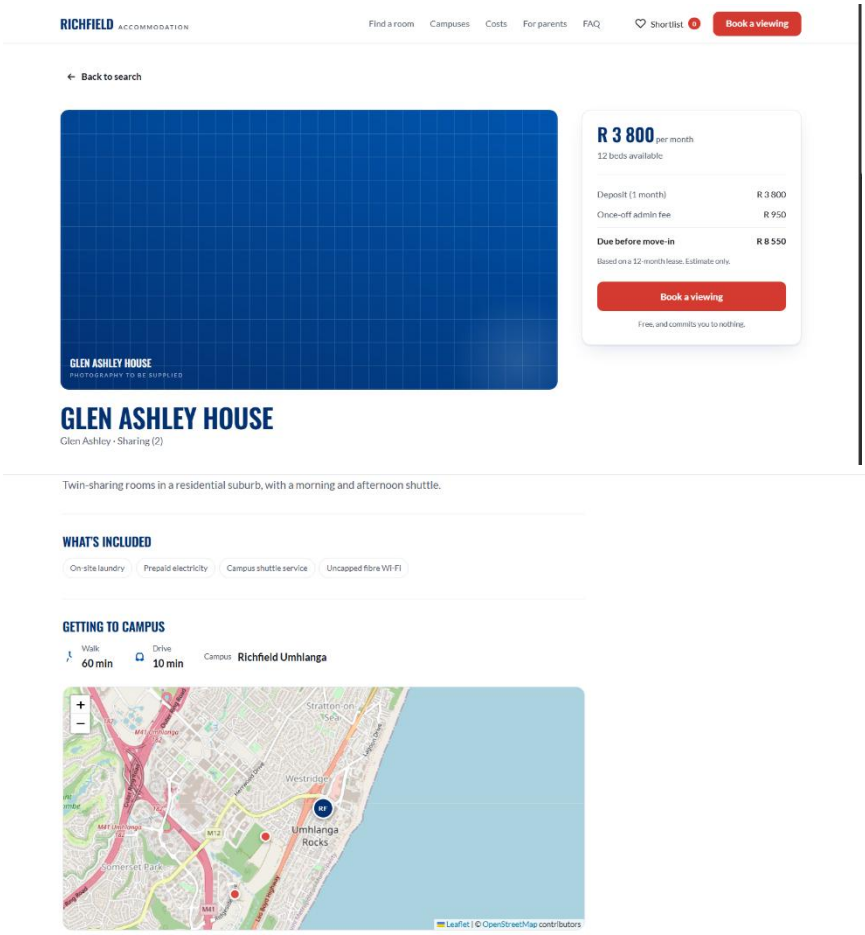


Figure 4.13 — Viewing-request form (apply.php)

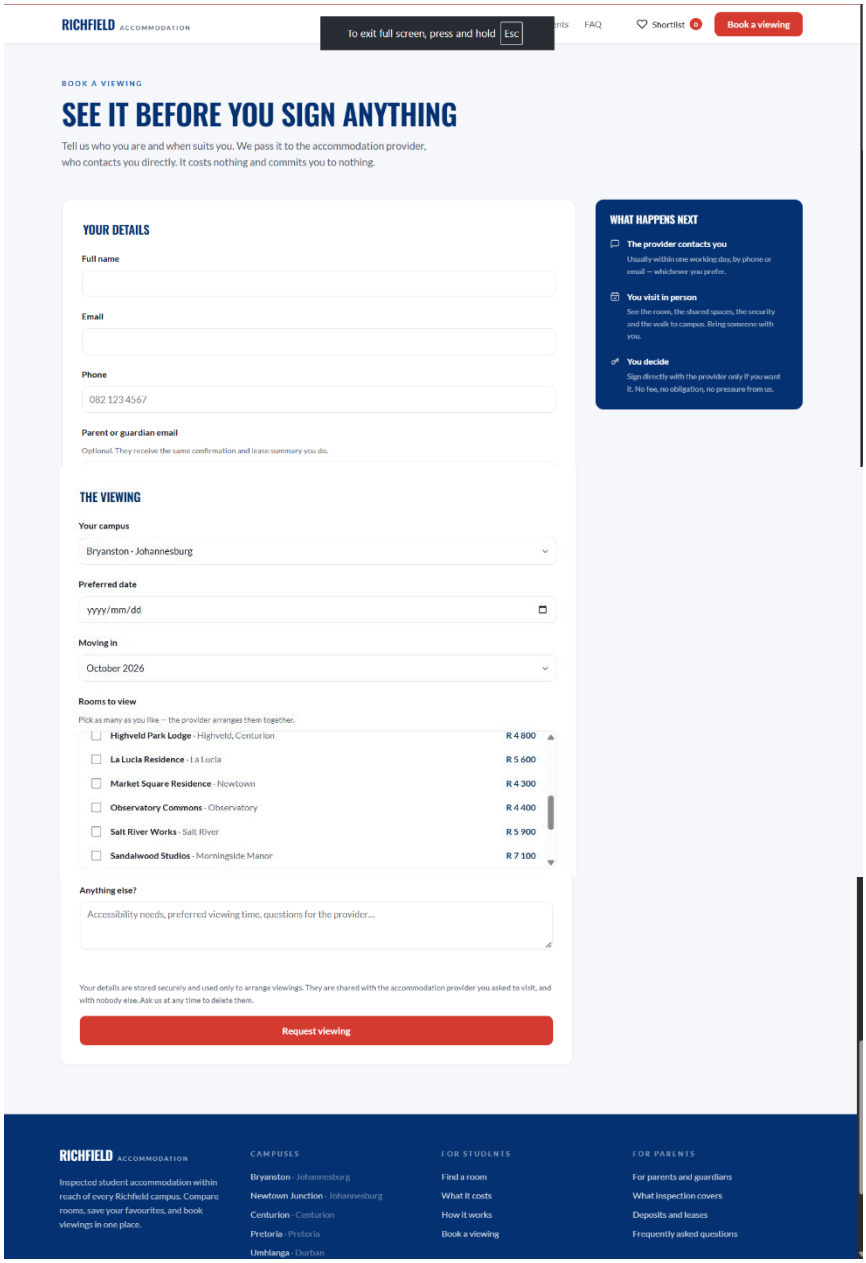


Figure 4.14 — Server-side validation errors

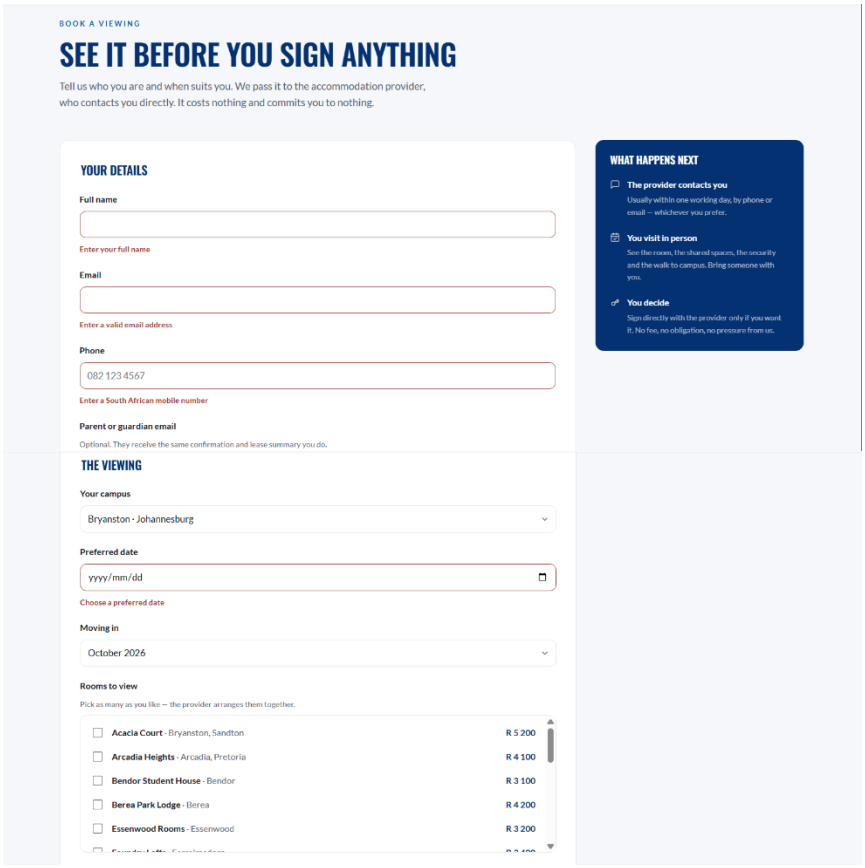
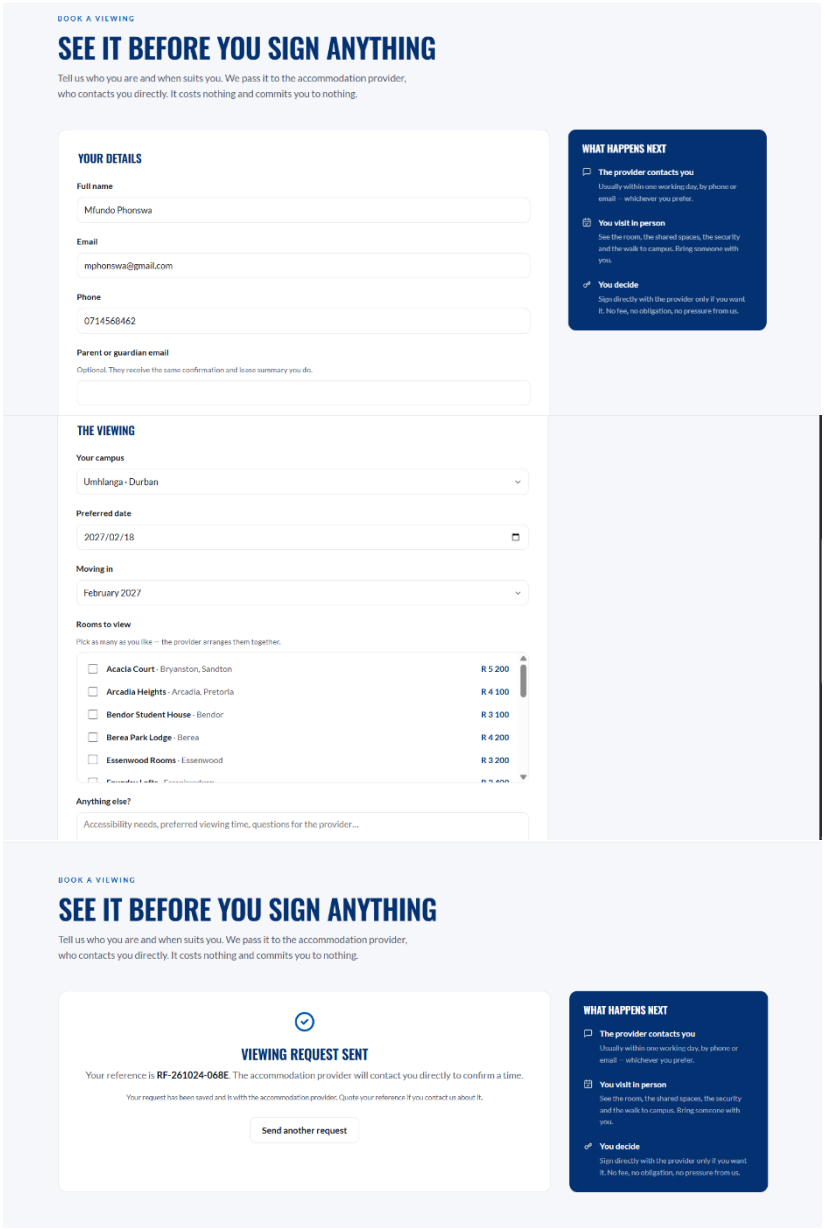


Figure 4.15 — Rooms to view, no error shown



Figure 4.16 — Confirmation with reference



4.7.4 Accessibility and responsive design

Every functionality must have the capability to be operated through Keyboard Input. Two of the brand colours did not work and needed some accessible derivatives; these are inserted into the stylesheet using their measured ratios rather than eye adjustments.

Table 4.7.4 — Contrast remediation

Colour	Measured	Required	Design response
White on brand red `#de473c`	4.14 : 1	4.5 : 1	A darker derivative `--rf-red-ink` (`#d43a2f`, 4.72 : 1) is used wherever white text sits on red; the original is retained for decoration only
`--rf-gray-light` on white	1.41 : 1	3 : 1	`--rf-numeral` (`#7d8896`, 3.60 : 1) is used for display numerals and icons

Responsive design is fluid instead of being driven by breakpoints. It is checked at seven widths 360 pixels, 390 pixels, 414 pixels, 768 pixels, 1024 pixels, 1280 pixels and 1920 pixels which requires zero horizontal overflow at each. Other accessibility provisions included are: skip-to-content link, honours prefers-reduced-motion, the equivalent listing of the map that is keyboard-navigable, and role="status" on confirmation messages so they’re announced.

All visual appearances are defined in one :root block with custom properties. The literal colour is only present in one other file, so to re-skin for another institution you just edit that block, the wordmark partial and the sample data and nothing else.

4.8 Security and Backup Design

4.8.1 Threat model

Design is from the value of the attacker. The system contains no money, no credentials and no accounts, which takes out whole classes of risk – but it does contain names, email addresses and phone numbers of students, which constitutes regulated personal information and the asset worth protecting.

Table 4.8.1 — Threats and design responses

Threat	Likelihood	Impact	Design response
SQL injection via URL or form parameters	High	Critical	Prepared statements exclusively; no concatenation of input into SQL anywhere (NFR2)
Cross-site scripting through stored or reflected content	Medium	High	Every dynamic value escaped through `e()` on output (NFR8)
Cross-site request forgery creating junk requests	Medium	Medium	Per-session token compared with `hash_equals()` (FR9)
Automated form submission flooding the table	High	Medium	**Not yet mitigated** — rate limiting is outstanding work (4.8.5)
Disclosure of personal data from the database	Low	Critical	Minimal collection (BR-12); credentials git-ignored; least-privilege database account in production
Information disclosure through error messages	Medium	Medium	Errors logged, never displayed; students see a generic message
Accidental commit of credentials or a data dump	Medium	Critical	`config.php` and `*.sqlite` git-ignored; a committed sample template instead

Search engines indexing an unapproved branded site	High	Medium	`noindex` on every page plus a not-affiliated footer notice

4.8.2 Security controls

Table 4.8.2 — Controls as designed

Control	Design	Verified by
Injection prevention	PDO with `ATTR_EMULATE_PREPARES = false`, so parameters travel separately from the statement and can never be parsed as SQL	Phase 5 security layer — injection payloads against every parameter
Output escaping	`htmlspecialchars` with `ENT_QUOTES | ENT_SUBSTITUTE`, UTF-8, on every dynamic value	Script payload in a form field returns as literal text
CSRF protection	32 random bytes per session, hex-encoded, compared in constant time	Submission with the token removed is refused
Error handling	`ERRMODE_EXCEPTION`; exceptions caught at the capture point, logged via `error_log`, replaced with a neutral message	Database stopped mid-submission produces guidance, not a stack trace
Credential handling	Configuration outside version control; a sample template committed in its place	Repository inspection

Transactional integrity	Both inserts in one transaction, rolled back entirely on any exception	Induced failure leaves no partial row
Data minimisation	No identity numbers, no banking details, no funding documents, no street addresses	Schema review

4.8.3 Data protection and POPIA

The *Protection of Personal Information Act 4 of 2013* applies to the names, email addresses, phone numbers and guardian email addresses held in `viewing_requests`. The design addresses four of its conditions directly.

Table 4.8.3 — POPIA conditions and design provisions

Condition	Provision
Minimality	Only what arranging a viewing requires. Specifically excluded: identity numbers, date of birth, student number, banking details, funding documents and street addresses.
Purpose specification	Stated above the submit button, in the student's own view, at the moment of collection — not buried in a policy page.
Openness	The notice names who the data is shared with (the accommodation provider the student asked to visit) and states that deletion may be requested at any time.
Retention	**Not yet satisfied.** No retention period has been agreed and no automated deletion exists. The design provides for it: `created_at` plus the CASCADE on `viewing_request_rooms` make a scheduled purge a single DELETE. The period must be agreed with the client before go-live.

A deletion request is satisfied by deleting the `viewing_requests` row; the CASCADE removes the linked rooms automatically. This is why that particular cascade is retained while the one on `listing_slug` is changed to RESTRICT — one exists to erase personal data on request, the other would have erased history by accident.

4.8.4 Backup and recovery design

Reference data is reproducible from `seed.php` and needs no backup. Captured data cannot be reproduced and is the only thing whose loss would matter.

Table 4.8.4 — Backup design

Aspect	Design
Scope	`viewing_requests` and `viewing_request_rooms`. Everything else is regenerable from source.
Method	`mysqldump` of the schema, written to a timestamped file. SQLite deployments copy the single database file.
Frequency	Nightly in production; before each demonstration and before any schema change during assessment.
Retention	Seven daily, four weekly. Backups contain personal data, so they fall under the same retention policy as the live tables.
Storage	Off the web host, encrypted at rest. **Never in the Git repository** — `.gitignore` blocks the obvious filenames, but it cannot stop a rename, so this is a procedural control as much as a technical one.
Restore procedure	`mysql -u root richfield_accommodation < backup.sql`, followed by a row count against the backup's own record.

Verification	A backup that has never been restored is an assumption. One restore into a scratch schema per month, confirming row counts and a spot-checked reference.
Recovery objectives	RPO 24 hours (one nightly cycle). RTO 1 hour — the application is a file copy and a dump import.

4.8.5 Outstanding security work

Recorded here deliberately. A design document that lists only what was done tells a maintainer nothing about where the edges are.

Table 4.8.5 — Required before public deployment

No.	Item	Why it matters
1	**Rate limiting on the viewing form**	A public form that writes rows with no throttle can be flooded by a script. A per-IP hourly cap is the minimum.
2	**Move application files above the web root**	`includes/` and `database/` currently sit inside the published directory (4.4.1). Only the page files and `assets/` should be reachable.
3	**Agree and implement a POPIA retention period**	Personal data currently accumulates indefinitely. The mechanism is designed; the policy decision is outstanding.
4	**HTTPS in production**	The form carries personal data. Without TLS it crosses the network in clear text.
5	**Least-privilege database account**	The development configuration uses `root` with a blank password — acceptable on a local XAMPP install, unacceptable anywhere else.

6	**Administrative interface**	Requests arrive with a `status` column that nothing reads. Until an interface exists, staff must query the database directly.
7	**Email delivery**	The guardian address is stored but never used. Section 4.7.2 corrects the wording in the meantime; the two must change together.

Conclusion

The Phase 4 analysis turns the Phase 3 analysis into a specification that can be built upon. This architecture is made up of four layers with one-directional dependencies. This architecture satisfies two rules which can be checked by inspection. No page contains SQL. Only the connector opens a connection. The nine tables in Third Normal Form of the database design have a chosen deletion behaviour per relationship. The specification of program design outlines the five algorithms with business significance, while the specification of interface design specifies all input, validation and error message.

All six integrity findings from Phase 3 §3.4.5 are resolved by design and are traceable through Table 4.1.2 to the section that resolves each one. A seventh item — the inaccurate guardian-email wording — is corrected in 4.7.2 under NFR5.

The design also documents what still needs to be completed. According to Table 4.8.5, there are seven items that must be adhered to to transform this prototype into a system able to safely hold real students' personal information; probably the most significant ones are implementing rate limiting and moving the app files above the web root and agreeing a POPIA retention period. Having these criticisms does not mark a flaw in the design – their absence would..

Phase 5 implements and tests against this specification. The validation sequence in 4.6.1, the integrity rules in 4.5.4 and the input specification in 4.7.2 are the acceptance criteria that the Phase 5 test cases will be written from.

References

Republic of South Africa (2013) *Protection of Personal Information Act 4 of 2013*. Pretoria: Government Printer.

World Wide Web Consortium (2023) *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. Available at: https://www.w3.org/TR/WCAG22/ (Accessed: 24 October 2026).

Group 55 (2026) *EMERGEX Phase 2 — Planning*. IT Project 700, Richfield Graduate Institute of Technology.

Group 55 (2026) *EMERGEX Phase 3 — Analysis*. IT Project 700, Richfield Graduate Institute of Technology.

Project source: Group 55 repository `emergex-group-55`, directory `Richfield accommodation interface`.